

A search reduction from 3-CNF satisfiability to output-line fault detection

Reconstruction for expert review
25 September 2026

Abstract

Detecting both stuck-at polarities on a circuit output asks whether that output can take both Boolean values. We turn a three-literal CNF formula φ into the single-output circuit $w = z \wedge \varphi(x)$, where z is a fresh input. The stuck-at-1 fault always has a test, while a stuck-at-0 test exists exactly when φ is satisfiable. Every complete target test set therefore yields a satisfying assignment, and the target's no-solution answer decodes to source no-solution. The construction uses at most $6m$ gates for $m \geq 1$ clauses and has polynomial bit complexity even when the variable count is binary encoded. This is an explicit search reconstruction of a classical output-fault hardness mechanism.

1. Introduction

A stuck-at fault forces one circuit signal to a fixed bit. A test detects the fault when the faulty and fault-free circuits give different outputs on the same input. Ibarra and Sahni identified output-line detectability with nonconstancy of the circuit function in their Theorem 3.4(1) [1]. Their hardness proof uses a Boolean source framed through 3-DNF tautology. Here the source is the search problem for three-literal CNF, and the target must return tests for both polarities or an explicit no-solution answer. The reduction must recover a source output from every valid target output.

The main result supplies that complete rule. A fresh input z makes the circuit output zero regardless of the formula, while setting $z = 1$ exposes its satisfying assignments. The same final wire is both observed and faulted. Figure 1 shows the signal path.

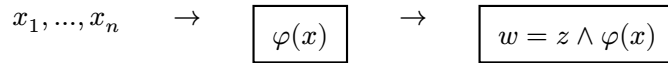


Figure 1: The formula circuit feeds the final AND gate together with the independent input z . The final wire w is the sole observed output and sole designated fault location; either stuck-at polarity replaces its value at the output.

2. Problems and encoding

A source instance consists of a nonnegative variable count n and a list of m clauses. Each clause has exactly three signed, nonzero indices with absolute value at most n . Repeated literals are allowed; an empty clause list denotes the true formula. A valid source output is a length- n bit vector satisfying every clause, or NO-SOLUTION precisely when no such vector exists. We write its valid-output set as $S_{A(\varphi)}$.

A target instance is an acyclic Boolean circuit represented by its input count and a topologically ordered list of fan-in-two AND/OR gates and fan-in-one NOT gates. Inputs and gate outputs have consecutive wire indices. Lists identify observed outputs and designated fault wires. A single stuck-at- b fault replaces the signal on its designated wire by b before downstream use or output observation. A target output contains one detecting input vector for each designated wire and each polarity $b \in \{0, 1\}$. If any required fault has no detecting vector, its only valid output is NO-SOLUTION. We write the target valid-output set as $S_{B(C)}$. The order and choice of detecting vectors are unrestricted.

The input count is a numeric field; an input wire is indexed without a separate gate record. This is the gate-list convention used by the executable contract. A complete target output nevertheless writes every bit of each test vector explicitly. These encoding facts matter when n is much larger than the bit length of its numeric representation.

3. Construction and recovery

For each source φ , introduce one fresh input z . Build a gate for each used negative literal, a two-gate OR tree for each clause, and an AND tree joining the clauses. For $m = 0$, use $z \vee \neg z$ for the true formula signal. Append one AND gate computing

$$w(x, z) = z \wedge \varphi(x). \tag{1}$$

Make its output wire the sole observed output and the sole designated fault wire. The algorithm names wires in gate-list order, so all gate inputs refer to earlier wires. It returns a legal target circuit $F(\varphi)$.

Recovery $G(\varphi, y)$ returns NO-SOLUTION when y is the target no-solution answer. Otherwise, it reconstructs the final wire index from φ , locates the stuck-at-0 entry in y , and returns the first n bits of that entry's input vector. Reconstructing the index uses only φ ; it does not rely on state saved by the forward process.

4. Correctness

Lemma 1 (output-fault test). If a wire w is both a circuit output and the fault location, a stuck-at- b fault is detected by an input precisely when the fault-free value of w equals $1 - b$.

Proof. Under the fault, the observed value is b . Without the fault, the observed value is w . The two outputs differ exactly when $w \neq b$.

Lemma 2 (the two polarities). In $F(\varphi)$, stuck-at-1 is always detectable. Stuck-at-0 is detectable if and only if φ has a satisfying assignment.

Proof. Taking $z = 0$ makes $w = 0$ in Equation 1, so Lemma 1 supplies a stuck-at-1 test. A stuck-at-0 test exists exactly when $w = 1$ on some input. Equation 1 gives $w = 1$ exactly when $z = 1$ and $\varphi(x) = 1$. The argument includes an empty clause list.

Theorem (search reduction). For every legal three-literal CNF φ and every $y \in S_{B(F(\varphi))}$, the output $G(\varphi, y)$ belongs to $S_{A(\varphi)}$.

Proof. If φ is unsatisfiable, Lemma 2 makes stuck-at-0 undetectable. The target semantics then admit exactly NO-SOLUTION, and G returns the valid source no-solution answer. If φ is satisfiable, Lemma 2 makes both polarities detectable, so every valid target output is a complete two-entry test set. Its stuck-at-0 entry detects a fault on the output wire. Lemma 1 and Equation 1 imply that the corresponding input has $\varphi(x) = 1$. Recovery returns this x , independently of entry order and test-vector choice. Both cases have at least one valid target output, so the implication is not vacuous.

5. Bit complexity

Let L be the bit length of the encoded source and m its number of clauses. At most $3m$ distinct negative literals are used. For $m \geq 1$, the construction uses at most $3m$ NOT gates, $2m$ clause OR gates, $m - 1$ joining AND gates, and one final AND gate: at most $6m$ gates. For $m = 0$, it uses three gates. Each wire index takes $O(\log(n + m + 2))$ bits. Thus the target encoding has $O((m + 1) \log(n + m + 2))$ bits. Since $m \leq L$ and $\log(n + 1) = O(L)$, construction and serialization take polynomial bit time in L .

Recovery reconstructs this compact circuit and scans its target output y . For a complete output, y contains explicit input vectors of length $n + 1$, so copying the recovered n bits is bounded by $|y|$. For NO-SOLUTION, recovery writes a fixed string. Its running time and output length are polynomial in $L + |y|$. Both maps are deterministic and call no solver. The target solver's work is outside these bounds.

6. Relation to earlier work

Ibarra and Sahni's output-line result [1] states the nonconstancy criterion for detecting both output faults. The construction above uses that criterion with a fresh gating input to connect the fixed 3-CNF search source to complete target test sets. It gives an explicit decoder and treats the no-solution case. It does not establish a new hardness theorem or a first use of the gating step. The original paper also studies finding test sets under an irredundancy promise in Theorem 3.5; our target instead has an explicit no-solution output when a required fault is undetectable.

7. Conclusion

The gated formula output gives a total polynomial-time instance map and recovery map for the stated fault model. Its proof handles both stuck-at polarities, satisfiable and unsatisfiable formulas, and every valid target output. The target solver can still require exponential work, and the reduction makes no practical speed claim.

References

- [1] O. H. Ibarra and S. K. Sahni, "Polynomially Complete Fault Detection Problems," *IEEE Transactions on Computers* C-24(3), 242–249 (1975). Section II, problem P4; Theorem 3.4(1) and Theorem 3.5, Case 4. Primary-source PDF.

A. Verification and reproducibility

The executable source and target encodings are specified in `contract.md` the two maps are in `algorithm.py`. Python 3.12.14, uv 0.12.17 and Z3 5.1.0 were used for the prepared corpus. Typst 0.15.1 compiled this manuscript. Run from the repository root:

```
uv sync --locked
```

```
uv run python campaigns/three-sat-circuit-fault-detection/work/check.py --self-test
```

```
uv run python campaigns/three-sat-circuit-fault-detection/work/check.py --candidate  
campaigns/three-sat-circuit-fault-detection/work/algorithm.py
```

```
uv run python campaigns/three-sat-circuit-fault-detection/work/verify.py --candidate  
campaigns/three-sat-circuit-fault-detection/work/algorithm.py
```

```
uv run python campaigns/three-sat-circuit-fault-detection/reviews/001/check_large_input.py
```

```
uv run python campaigns/three-sat-circuit-fault-detection/reviews/002/  
check_repaired_cli.py
```

The prepared corpus has 115 source formulas: 100 seeded random formulas of three to six variables and 15 hand-designed edge cases of zero to four variables. Z3 independently labels satisfiability; exhaustive fault simulation solves each generated target with at most seven inputs. The candidate loop checked 201 valid target outputs. A second checker, using separate exhaustive source and target solvers, checked 123 more source formulas with one to five variables and 243 target outputs, including alternate complete test sets. The last two commands exercise a 4,301-digit input count at the CLI boundary. These are finite checks; the theorem above supplies the unrestricted correctness argument.

The default `algorithm.py` mode reads one source JSON object from standard input and writes the target JSON. Its `--extract` mode reads a JSON object with keys `source` and `target_solution` and writes the recovered source output. The two modes may run in separate fresh processes. No command uses a timeout.